

Tutorial

MLP Forward and Backward Pass Computation

Course:

Deep Learning

By:

Dr. Zohair Ahmed

Slope tells us **how something is changing**.

1. Slope tells us how steep something is

Steep = goes up or down very quickly.

Imagine walking on a road:

- If the slope is **small**, the road is almost flat.
- If the slope is **big**, the road is very steep.

So slope answers:

“How much does it go up or down?”

2. Slope tells us how fast something is changing

Examples:

- How fast your **speed** is increasing
- How fast **temperature** is rising
- How fast **money** is growing
- How fast a **graph** goes up or down

Slope basically means: “Rate of change”

3. Slope helps us understand patterns

If slope is:

- **Positive** → graph is going up
- **Negative** → graph is going down
- **Zero** → graph is flat

This tells us what’s happening **even without numbers**.

4. Slope helps us predict the future

If a company’s profit graph has a positive slope, we know:

Profit is increasing → things are going well.

If slope is negative:

Profit is decreasing → something is wrong.

Slope helps in physics, economics, ML, engineering — everywhere something is **changing**.

Super Easy Example

If you gain:

- 2 kg every month → slope = +2
- lose 1 kg every month → slope = -1
- weight stays same → slope = 0

Slope simply tells **how fast and in which direction things change over time**.

One-line answer

We need slope to understand how fast something changes and in which direction it is going.

If slope works, why did we invent derivatives?

Because **slope only works for straight lines**, but **derivatives work for everything else**.

1. Slope works only when the line is straight

Slope = rise / run

This stays the **same everywhere** for a straight line.

Example:

A road going up at the same angle all the way → slope works fine.

2. But real-world lines are curves, not straight

Think about:

- A hill
- Speed of a car
- Temperature in a day
- Money profit graph
- Body weight over time
- Machine learning loss curve

NONE of these are straight.

They go up, down, bend, twist.

Slope can't describe a curve.

It gives only ONE number.

Derivative gives this: 'How much does it change right here, right now?'

1. Your car's speedometer = Derivative in real life

When you look at your car's speedometer:

- It shows **your speed at this exact moment**, not average speed.
- If you press the accelerator, the number changes immediately.

This is **instantaneous change**.

This is exactly what a derivative tells you.

It tells how fast something is changing **right now**, not over a long gap.

A normal slope cannot do this because it needs TWO points → past and future.

But derivative uses teeny tiny points so close they are almost touching → gives NOW.

2. Thermometer example

Imagine your thermometer says:

- **37.2°C at this second**

How does it know “this second”?

Because it is measuring **instant change in temperature**.

That’s what a derivative does to a graph.

👉 It says: “What is the slope *precisely at this point*?”

3. Standing on a hill example

You are standing on a hill.

Someone asks:

“How steep is the ground right under your feet?”

You don’t need to look at the whole hill.

You just look **down at your feet**.

If it feels like:

- flat → slope = small
- very slanted → slope = large
- straight drop → slope = huge

This “under your feet steepness” = derivative.

Why simple slope cannot tell ‘right now’

Simple slope = needs 2 points

Derivative = needs 2 points that are EXTREMELY close

→ So close that they behave like **1 single point**

→ So we know the slope at that point.

That’s how math creates the “right now” idea.

Super Simple Explanation

- Slope tells **average change**:
“Between yesterday and today, your weight changed 1 kg.”
- Derivative tells **instant change**:
“Right now this very moment, the graph is going up fast.”

A derivative tells “right now” because it measures change using points that are extremely close together, so it captures the exact change at that single moment.

A partial derivative means:

👉 Change in one direction while keeping the other things fixed.

It’s like asking:

“How does the output change if I change just ONE variable, and keep all others the SAME?”

What is the Chain Rule

The **Chain Rule** helps you find the derivative when a function is **inside another function**.

Think of it like:

If a machine is inside another machine, and the inside one changes, the outside one also changes.

Chain Rule tells how the whole thing changes.

Easiest Real-Life Explanation: “Burger Wrapper Idea”

Imagine a burger wrapped in paper.

- The **burger** = inner function
- The **wrapper** = outer function

If the **burger size changes**, the **wrapper size must change too**.

Chain Rule tells:

- 👉 How the wrapper changes
- 👉 Because the burger changed

1. Constant Rule

- Rule: The derivative of a constant is $0 \rightarrow \frac{d}{dx}[c] = 0$
Example: $\frac{d}{dx}[7] = 0$

2. Power Rule

- **Rule:** For x^n : $\frac{d}{dx}[x^n] = nx^{n-1}$
- **Example:** $\frac{d}{dx}[x^5] = 5x^4$

3. Composite function: A **composite function** means one function is *inside* another: $f(x) = g(h(x))$

Example: $f(x) = (3x + 1)^5$

Here:

- **Inner function:** $h(x) = 3x + 1$
- **Outer function:** $g(u) = u^5$

So $f(x) = g(h(x))$.

3.1. Chain Rule (Simple Explanation)

If:

$$y = g(h(x))$$

Then the derivative is:

$$y' = g'(h(x)) \cdot h'(x)$$

In words:

Differentiate the outer function, keep the inside same, then multiply by derivative of the inside.

Example 1: Decompose and Differentiate

Function: $y = (2x^2 + 5)^4$

Step 1: Decompose

- Inside function $\rightarrow h(x) = 2x^2 + 5$

- Outside function $\rightarrow g(u) = u^4$

Step 2: Apply Chain Rule

Outer derivative: $g'(u) = 4u^3$

Inner derivative: $h'(x) = 4x$

Step 3: Combine: $y' = 4(2x^2 + 5)^3 \cdot 4x$

Final Answer: $y' = 16x(2x^2 + 5)^3$

4 What is a Partial Derivative? (Simple Explanation)

When a function has **two or more variables**, like: $f(x, y)$

A **partial derivative** means: Differentiate with respect to **one variable**, while treating **all other variables as constants**.

Partial derivative w.r.t. x : $\frac{\partial f}{\partial x}$

Partial derivative w.r.t. y : $\frac{\partial f}{\partial y}$

Given:

$$f(x, y) = x^2y + 3y$$

1. Partial Derivative w.r.t. x

Treat y as a constant.

Step 1: Differentiate x^2y

- y is constant $\rightarrow$ keep it as it is
- Derivative of x^2 is $2x$

$$\frac{\partial}{\partial x}(x^2y) = 2xy$$

Step 2: Differentiate $3y$

- $3y$ contains **no x** , so it is a **constant**
- Derivative of a constant is **0**

$$\frac{\partial}{\partial x}(3y) = 0$$

Final Answer:

$$\frac{\partial f}{\partial x} = 2xy$$

2. Partial Derivative w.r.t. y

Treat x as a constant.

Step 1: Differentiate x^2y

- x^2 is constant
- Derivative of y is 1

$$\frac{\partial}{\partial y}(x^2y) = x^2$$

Step 2: Differentiate $3y$

- Derivative of $3y$ w.r.t. y is 3

$$\frac{\partial}{\partial y}(3y) = 3$$

Final Answer:

$$\frac{\partial f}{\partial y} = x^2 + 3$$

Final Results (Complete)

$$\frac{\partial f}{\partial x} = 2xy$$

$$\frac{\partial f}{\partial y} = x^2 + 3$$

5 Partial Derivative and Chain Rule

They come together when:

The function has multiple variables (**Partial**) AND

The function is a composite function (function inside another) **Chain Rule**

Example

$$f(x, y) = (xy)^2$$

STEP 1 — Decompose

Let:

$$u = xy$$

Then the function becomes:

$$f = u^2$$

STEP 2 — Chain Rule Formula

For any variable (x or y):

$$\frac{\partial f}{\partial x} = \frac{df}{du} \cdot \frac{\partial u}{\partial x}$$

$$\frac{\partial f}{\partial y} = \frac{df}{du} \cdot \frac{\partial u}{\partial y}$$

Now compute each piece.

STEP 3 — Compute df/du

Outer function:

$$f = u^2$$

Power rule:

$$\frac{df}{du} = 2u$$

STEP 4 — Compute partials of the inner function

$$u = xy$$

With respect to x:

$$\frac{\partial u}{\partial x} = y$$

With respect to y:

$$\frac{\partial u}{\partial y} = x$$

STEP 5 — Combine using chain rule

Partial derivative w.r.t. x

$$\frac{\partial f}{\partial x} = \frac{df}{du} \cdot \frac{\partial u}{\partial x}$$

Substitute values:

$$= 2u \cdot y$$

Replace $u = xy$:

$$\frac{\partial f}{\partial x} = 2(xy)y$$

Simplify:

$$\boxed{\frac{\partial f}{\partial x} = 2xy^2}$$

Partial derivative w.r.t. y

$$\frac{\partial f}{\partial y} = \frac{df}{du} \cdot \frac{\partial u}{\partial y}$$

Substitute:

$$= 2u \cdot x$$

Replace $u = xy$:

$$\frac{\partial f}{\partial y} = 2(xy)x$$

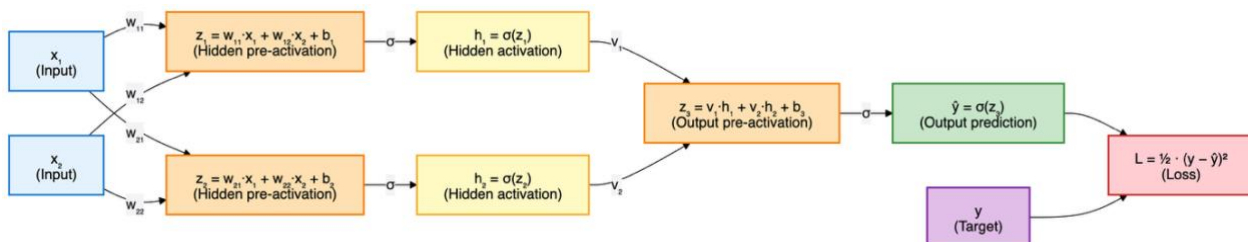
Simplify:

$$\frac{\partial f}{\partial y} = 2x^2y$$

FINAL ANSWERS

$$\frac{\partial f}{\partial x} = 2xy^2$$

$$\frac{\partial f}{\partial y} = 2x^2y$$



Weight	Meaning
w_{11}	first hidden neuron (h_1) ← first input (x_1)
w_{12}	first hidden neuron (h_1) ← second input (x_2)
w_{21}	second hidden neuron (h_2) ← first input (x_1)
w_{22}	second hidden neuron (h_2) ← second input (x_2)

1. **x_1 connects to both hidden neurons** via w_{11} (h_1) and w_{21} (h_2)
2. **x_2 connects to both hidden neurons** via w_{12} (h_1) and w_{22} (h_2)
3. **Hidden neurons connect to output** via v_1 ($h_1 \rightarrow z_3$) and v_2 ($h_2 \rightarrow z_3$)

4. **Biases** add directly to each neuron input

Update rule:

$$v_1^{\text{new}} = v_1^{\text{old}} - \eta \cdot \frac{\partial L}{\partial v_1}$$

Network Architecture Summary

Given Parameters:

Layer	Parameter	Value
Hidden H1	w_{11}	0.1
Hidden H1	w_{12}	0.2
Hidden H1	b_1	0.1
Hidden H2	w_{21}	0.3
Hidden H2	w_{22}	0.4
Hidden H2	b_2	0.1
Output	v_1	0.5
Output	v_2	0.6
Output	b_3	0.2

Activation function: Sigmoid for all neurons

Input: $x_1 = 1$, $x_2 = 0$

Target: $y = 1$

Loss function: $L = \frac{1}{2}(y - \hat{y})^2$

PART 1: FORWARD PASS (Complete Calculations)

Step 1: Hidden Layer Neuron H_1

Pre-activation z_1 (Linear Combination)

MULTIVARIABLE FUNCTION ALERT: This is a multivariable function because z_1 depends on two inputs: x_1 and x_2

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

Substitute values: $z_1 = (0.1)(1) + (0.2)(0) + 0.1$ $z_1 = 0.1 + 0 + 0.1$ $z_1 = 0.2$

Post-activation h_1 (Sigmoid)

COMPOSITION ALERT: $h_1 = \sigma(z_1)$ is a composition of functions: outer function σ , inner function z_1

$$h_1 = \sigma(z_1) = \frac{1}{1 + e^{-z_1}}$$

Substitute $z_1 = 0.2$: $h_1 = \frac{1}{1 + e^{-0.2}}$

Calculate step by step:

- $e^{(-0.2)} = 0.81873$
- $1 + 0.81873 = 1.81873$
- $1 \div 1.81873 = 0.54983$

$$h_1 \approx 0.5498$$

Step 2: Hidden Layer Neuron H_2

Pre-activation z_2 (Linear Combination)

MULTIVARIABLE FUNCTION ALERT: z_2 depends on both x_1 and x_2

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

Substitute values: $z_2 = (0.3)(1) + (0.4)(0) + 0.1$ $z_2 = 0.3 + 0 + 0.1$ $z_2 = 0.4$

Post-activation h_2 (Sigmoid)

COMPOSITION ALERT: $h_2 = \sigma(z_2)$ is a composition

$$h_2 = \sigma(z_2) = \frac{1}{1 + e^{-z_2}}$$

Substitute $z_2 = 0.4$: $h_2 = \frac{1}{1 + e^{-0.4}}$

Calculate step by step:

- $e^{(-0.4)} = 0.67032$
- $1 + 0.67032 = 1.67032$
- $1 \div 1.67032 = 0.59869$

$$h_2 \approx 0.5987$$

Step 3: Output Layer

Pre-activation z_3 (Linear Combination)

MULTIVARIABLE FUNCTION ALERT: z_3 depends on both h_1 and h_2

$$z_3 = v_1h_1 + v_2h_2 + b_3$$

Substitute values: $z_3 = (0.5)(0.5498) + (0.6)(0.5987) + 0.2$ $z_3 = 0.2749 + 0.3592 + 0.2$ $z_3 = 0.8341$

Post-activation $\hat{y}$ (Sigmoid)

COMPOSITION ALERT: $\hat{y} = \sigma(z_3)$ is a composition

$$\hat{y} = \sigma(z_3) = \frac{1}{1 + e^{-z_3}}$$

Substitute $z_3 = 0.8341$: $\hat{y} = \frac{1}{1 + e^{-0.8341}}$

Calculate step by step:

- $e^{(-0.8341)} = 0.43410$

- $1 + 0.43410 = 1.43410$
- $1 \div 1.43410 = 0.69729$

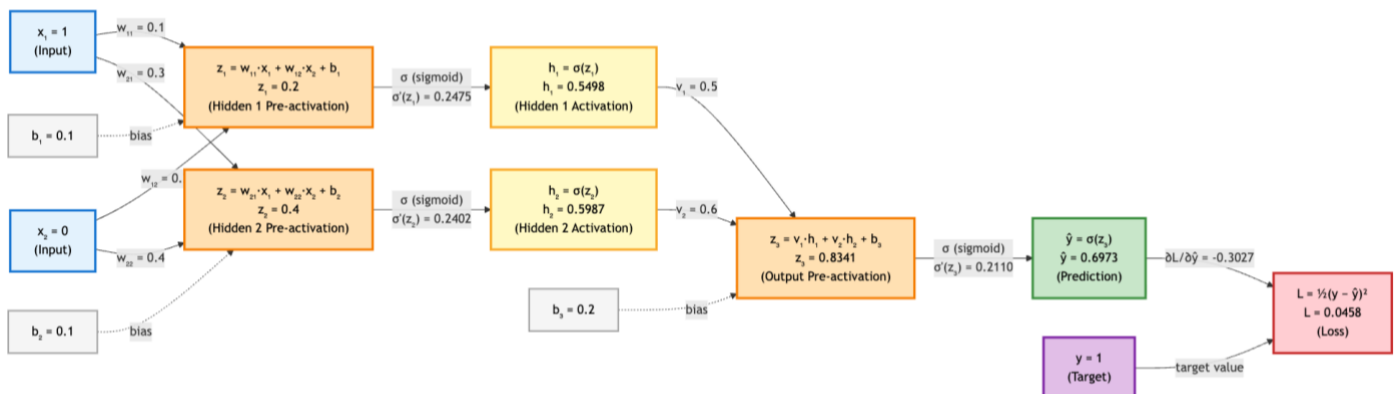
$$\hat{y} \approx 0.6973$$

Step 4: Loss Calculation

COMPOSITION ALERT: Loss is a composition: $L = f(y - \hat{y})$ where $f(u) = \frac{1}{2}u^2$

$$L = \frac{1}{2}(y - \hat{y})^2$$

Substitute values: $L = \frac{1}{2}(1 - 0.6973)^2$ $L = \frac{1}{2}(0.3027)^2$ $L = \frac{1}{2}(0.0916)$ $L \approx 0.0458$



PART 2: BACKWARD PASS (Complete Calculations)

$L \rightarrow \hat{y} \rightarrow z_3 \rightarrow \{h_1, h_2\} \rightarrow \{z_1, z_2\} \rightarrow \{\text{weights \& biases}\}$

NOW LET'S TRACE THE COMPLETE BACKPROP PATH

Perfect 🧐 this is the right way to learn it.

You already know:

- Composite functions → **Chain Rule**
- Multiple variables → **Partial derivatives**
- Forward pass values (great 👍)

After Forward Pass Values

$$z_1 = 0.2; h_1 = 0.5498; z_2 = 0.4; h_2 = 0.5987; z_3 = 0.8341; y = 0.6973; L = 0.0458$$

We will go **one tiny step at a time**.

STEP 1 — Start From the Loss

We begin at the very top of the graph.

Loss:

$$L = \frac{1}{2}(y - \hat{y})^2$$

You know:

- $y = 1$
- $\hat{y} = 0.6973$

But **don't compute anything yet**.

? Question 1

Is L a function of:

- A) only y
- B) only $\hat{y}$
- C) both
- D) neither

Think carefully: during training, which variable changes?



correct: B) only $\hat{y}$

During training:

- y is constant (given target)
- $\hat{y}$ depends on weights
- So **L is a function of $\hat{y}$ only**

So we treat:

$$L(\hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

**STEP 2 — First Derivative**

Now we compute:

$$\frac{\partial L}{\partial \hat{y}}$$

This is just **normal single-variable calculus** (no chain rule yet).



Let's rewrite it clearly:

$$L = \frac{1}{2}(y - \hat{y})^2$$

Treat y as constant.

? Question 2

If:

$$f(u) = \frac{1}{2}(u)^2$$

then

$$\frac{df}{du} = ?$$

Just basic power rule. What is it?

✅ **answer is: u**

You have:

$$f(u) = \frac{1}{2} u^2$$

Apply the basic power rule:

- Power rule: $\frac{d}{du}(u^n) = nu^{n-1}$
- Constant multiple rule: pull constants outside the derivative.

So:

$$\frac{df}{du} = \frac{1}{2} \cdot \frac{d}{du}(u^2) = \frac{1}{2} \cdot 2u = u$$

✅ **Final answer:**

$$\frac{df}{du} = u$$

Now apply this idea carefully to our case.

We have:

$$L = \frac{1}{2}(y - \hat{y})^2$$

Here:

$$u = (y - \hat{y})$$

So by chain rule:

Now think carefully:

$$\frac{\partial L}{\partial \hat{y}} = u \cdot \frac{d}{d\hat{y}}(y - \hat{y})$$

چین رول کی زنجیر (The Chain of Responsibility)

اسے ایک مثال سے سمجھیں۔ فرض کریں ایک فیکٹری میں خراب پروڈکٹ بنی۔

• $y - \hat{y}$ صرف یہ بتاتا ہے کہ پروڈکٹ کتنی خراب ہے۔

• ڈیریویٹو اور چین رول یہ بتاتے ہیں کہ:

1. پیکنگ مشین کی کتنی غلطی تھی؟ $(\frac{\partial L}{\partial \hat{y}})$

2. مکسچر بنانے والے کی کتنی غلطی تھی؟ $(\frac{\partial \hat{y}}{\partial z})$

3. کچا مال لانے والے کی کتنی غلطی تھی؟ $(\frac{\partial z}{\partial w})$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial w}$$

? Question 3

What is:

$$\frac{d}{d\hat{y}}(y - \hat{y})$$

Remember:

- y is constant
- derivative of $-\hat{y}$?

What do you get?

What is:

$$\frac{d}{d\hat{y}}(-\hat{y})$$

Is it:

- A) 1
- B) -1
- C) 0

✅ B) -1

(Optional) HOW to set up the chain rule

Chain rule formula is:

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

This means:

Step 1: Identify the outer function

The big function applied last.

Step 2: Identify the inner function (u)

The thing inside.

Step 3: Differentiate:

- outer derivative (keep inside as is)
- multiply by derivative of inner

Now apply chain rule step by step

$$\frac{dL}{d\hat{y}} = \frac{df}{du} \cdot \frac{du}{d\hat{y}}$$

Part 1: Differentiate the outer

$$f(u) = \frac{1}{2}u^2$$

$$\frac{df}{du} = u$$

Part 2: Differentiate the inner

$$u = y - \hat{y}$$

Derivative of y is constant

$$\frac{d}{d\hat{y}}(y) = 0$$

Derivative of $-\hat{y}$

$$\frac{d}{d\hat{y}}(-\hat{y}) = -1$$

Combine:

$$\frac{du}{d\hat{y}} = 0 + (-1) = -1$$

$$\frac{du}{d\hat{y}} = -1$$

Multiply them

$$\frac{dL}{d\hat{y}} = u \cdot (-1)$$

Substitute back:

$$u = (y - \hat{y})$$

$$\frac{dL}{d\hat{y}} = (y - \hat{y}) \cdot (-1)$$

$$\boxed{\frac{dL}{d\hat{y}} = \hat{y} - y}$$

✅ **Correct Answer (B) -1**

$$\frac{d}{d\hat{y}}(-\hat{y}) = -1$$

So now assemble both pieces.

We had:

$$\frac{\partial L}{\partial \hat{y}} = \underbrace{(y - \hat{y})}_{\text{outer}} \cdot \underbrace{(-1)}_{\text{inner}}$$

So:

$$\frac{\partial L}{\partial \hat{y}} = -(y - \hat{y})$$

Which we can rewrite as:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

(Just multiplying the negative inside.)

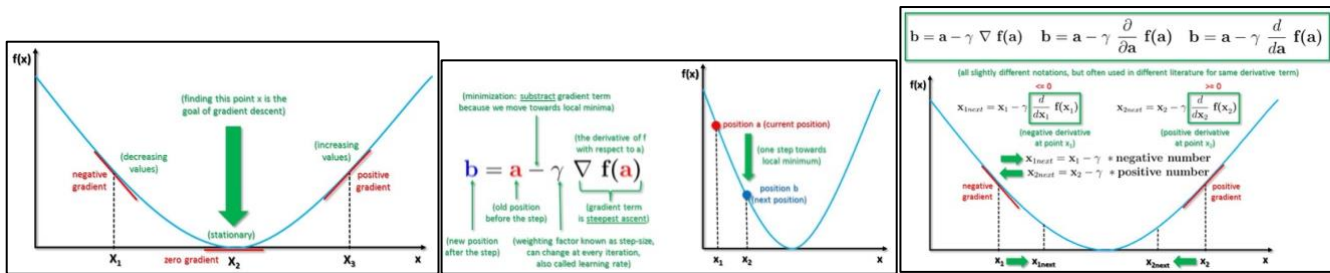
🔥 Important Insight

Notice something:

If prediction is **too small**,
 $\hat{y} < y \rightarrow$ derivative is negative $\rightarrow$ gradient descent will increase it.

If prediction is **too large**,
 derivative is positive $\rightarrow$ gradient descent decreases it.

So the math automatically pushes the output in the correct direction.



نتیجہ (Result)	گریڈینٹ ڈیسینٹ کا ایکشن	ڈیریویٹو کا سائن	صورتحال (Scenario)
Output بڑھ جاتی ہے	Weight کو بڑھاتا ہے	Negative	$\hat{y}$ چھوٹا ہے
Output کم ہو جاتی ہے	Weight کو کم کرتا ہے	Positive	$\hat{y}$ بڑا ہے

Now let's make it concrete.

You have:

- $\hat{y} = 0.6973$
- $y = 1$

? Compute this:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = ?$$

Just plug the numbers.

Perfect

$$\frac{\partial L}{\partial \hat{y}} = 0.6973 - 1 = -0.3027$$

Excellent.

What This Means Physically

The slope is **negative**.

That means:

If we slightly increase $\hat{y}$, the loss will **decrease**.

So the network needs to push the output **upward**.

Everything makes sense intuitively.

Why don't we just plug numbers directly into the loss and differentiate everything at once?
Why break it into chain rule pieces?

Core Idea

Because the weights are **not directly inside the loss formula**.

Look carefully:

$$L = \frac{1}{2} (y - \hat{y})^2$$

Do you see any weight here?

No.

The weights are hidden inside:

$$\hat{y} = \sigma(z_3)$$

and

$$z_3 = v_1 h_1 + v_2 h_2 + b_3$$

and

$$h_1 = \sigma(z_1)$$

and

$$z_1 = w_{11} x_1 + w_{12} x_2 + b_1$$

So the loss is a **function of a function of a function of a function**.

It is nested.

Important

If you want:

$$\frac{\partial L}{\partial w_{11}}$$

You cannot differentiate directly because:

$$L \neq \text{simple function of } w_{11}$$

It is:

$$L(\hat{y}(z_3(h_1(z_1(w_{11}))))))$$

That is exactly what **chain rule** is designed for.

Think of it Like This

If I ask:

“How does temperature affect crop yield?”

But:

Temperature → soil moisture → plant growth → crop yield

You must go step by step backward.

Same here.

STEP 3 — Now We Go One Level Back

We just computed:

$$\frac{\partial L}{\partial \hat{y}}$$

But the weights do NOT directly affect $\hat{y}$.

They affect:

$$z_3$$

And then:

$$\hat{y} = \sigma(z_3)$$

So now we apply **chain rule again**:

$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3}$$

Question

Since:

$$\hat{y} = \sigma(z_3)$$

What is the derivative of sigmoid?

You already know this formula.

What is:

$$\frac{d}{dz} \sigma(z)$$

?

We know the formula of Sigmoid but don't know Derivative

Now we're at the **heart of backprop**.

Let's derive it calmly — pure calculus.

You know:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Good.

Step 1 — Rewrite It in Power Form

Fractions are harder to differentiate.

Rewrite as:

$$\sigma(z) = (1 + e^{-z})^{-1}$$

Now it's something like:

$$f(z) = g(z)^{-1}$$

So this is again a **composite function**.

Outer function:

$$f(u) = u^{-1}$$

Inner function:

$$u = 1 + e^{-z}$$

Step 2 — Differentiate Outer Function

If:

$$f(u) = u^{-1}$$

What is:

$$\frac{df}{du}$$

Using power rule?

(Remember: bring power down, subtract 1.)

Your turn. Only this step.

Perfect 

$$\frac{d}{du}(u^{-1}) = -u^{-2}$$

Step 3 — Now Inner Derivative

We had:

$$u = 1 + e^{-z}$$

Now we compute:

$$\frac{du}{dz}$$

Derivative of 1 is 0.

So we only differentiate:

$$** \text{Derivative of the sigmoid Function: } \sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

*****It will calculate by Quotient rule and exponent***

and because in backprop:

$$\hat{y} = \sigma(z_3)$$

we simply rewrite:

$$\frac{d\hat{y}}{dz_3} = \hat{y}(1 - \hat{y})$$

Now fill Values:

$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_3}$$

Calculate:

$$\frac{\partial L}{\partial z_3} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

Step 1 — Compute $\hat{y} - y$

$$\hat{y} - y = 0.6973 - 1 = -0.3027$$

Step 2 — Compute $\hat{y}(1 - \hat{y})$

$$\begin{aligned}\hat{y}(1 - \hat{y}) &= 0.6973 \cdot (1 - 0.6973) = 0.6973 \cdot 0.3027 \\ 0.6973 \cdot 0.3027 &\approx 0.2112\end{aligned}$$

Step 3 — Multiply them together

$$\frac{\partial L}{\partial z_3} = -0.3027 \cdot 0.2112 \approx -0.064$$

Insight

- This value is the **error signal at the output neuron**.
- Negative $\rightarrow$ the neuron needs to **increase** z_3 (and hence $\hat{y}$) to reduce loss.

Next Step

We are now at the **weights connecting hidden $\rightarrow$ output**:

$$z_3 = v_1 h_1 + v_2 h_2 + b_3$$

We want:

$$\frac{\partial L}{\partial v_1}, \frac{\partial L}{\partial v_2}, \frac{\partial L}{\partial b_3}$$

Step 1 — Partial derivative formula

For any linear neuron:

$$z_3 = v_1 h_1 + v_2 h_2 + b_3$$

- Treat h_1, h_2 as constants.
- Then:

$$\frac{\partial z_3}{\partial v_1} = h_1, \frac{\partial z_3}{\partial v_2} = h_2, \frac{\partial z_3}{\partial b_3} = 1$$

Constant rule

If a constant multiplies a variable:

$$\frac{d}{dx}(c \cdot x) = c \cdot \frac{dx}{dx} = c \cdot 1 = c$$

- c is treated as constant
- x is the variable we're differentiating

Apply to $v_1 h_1$

Here:

- h_1 = constant
- v_1 = variable

$$\frac{\partial}{\partial v_1}(v_1 \cdot h_1) = h_1 \cdot \frac{\partial v_1}{\partial v_1} = h_1 \cdot 1 = h_1$$

✅ That's exactly why v_1 is treated as 1 — it's **the derivative of the variable w.r.t itself**.

Step 2 — Apply chain rule

$$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial v_1} = \delta_3 \cdot h_1$$

Step 2.1 — Identify the chain

- L does not directly depend on v_1
- But L depends on z_3 (output neuron input)
- And z_3 depends on v_1

So we have a **composite function**:

$$v_1 \rightarrow z_3 \rightarrow L$$

Chain rule tells us:

$$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial v_1}$$

Step 2. — Recognize the pieces

1. $\frac{\partial L}{\partial z_3}$
 - This is the “error signal” at the output neuron.
 - We called it $\delta_3 = \frac{\partial L}{\partial z_3}$
 - Already calculated: -0.064
2. $\frac{\partial z_3}{\partial v_1}$
 - $z_3 = v_1 h_1 + v_2 h_2 + b_3$
 - Partial derivative w.r.t $v_1 = h_1$ (as we discussed)

Step 3 — Multiply

Chain rule says **multiply these two**:

$$\frac{\partial L}{\partial v_1} = \delta_3 \cdot h_1$$

Step 4 — Intuition

- δ_3 = how much **loss changes if z_3 changes**
- h_1 = how much **z_3 changes if v_1 changes**
- Multiply → how much **loss changes if v_1 changes** ✓

Step 5 — Plug numbers (optional)

$$\delta_3 = -0.064, h_1 = 0.5498$$

$$\frac{\partial L}{\partial v_1} = -0.064 \cdot 0.5498 \approx -0.0352$$

Similarly:

$$\frac{\partial L}{\partial v_2} = \delta_3 \cdot h_2$$

$$\frac{\partial L}{\partial b_3} = \delta_3 \cdot 1 = \delta_3$$

Where:

$$\delta_3 = \frac{\partial L}{\partial z_3} = -0.064$$

Step 3 — Plug numbers

Forward pass values:

$$h_1 = 0.5498, h_2 = 0.5987$$

So:

$$\begin{aligned}\frac{\partial L}{\partial v_1} &= -0.064 \cdot 0.5498 \approx -0.0352 \\ \frac{\partial L}{\partial v_2} &= -0.064 \cdot 0.5987 \approx -0.0383 \\ \frac{\partial L}{\partial b_3} &= -0.064\end{aligned}$$

✅ Done.

🔥 Insight

- These gradients tell you **how to update the output layer weights**.
- Negative $\rightarrow$ increase weight (gradient descent: subtract $\eta \cdot \text{grad}$).

Backprop to Hidden Layer

We have:

Hidden neurons: h_1, h_2

Connections to output:

$$z_3 = v_1 h_1 + v_2 h_2 + b_3$$

- L depends on h_1 through z_3
- Similarly, L depends on h_2 through z_3

We want the gradients of **hidden weights**:

$$\frac{\partial L}{\partial w_{11}}, \frac{\partial L}{\partial w_{12}}, \frac{\partial L}{\partial b_1}$$

Step 1 — Apply chain rule carefully

Focus on w_{11} (weight from $x_1 \rightarrow h_1$):

$$w_{11} \rightarrow z_1 \rightarrow h_1 \rightarrow z_3 \rightarrow \hat{y} \rightarrow L$$

So by chain rule:

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}}$$

Step 2 — Identify each piece

1. $\frac{\partial L}{\partial z_3} = \delta_3 = -0.064$
2. $\frac{\partial z_3}{\partial h_1} = v_1 = 0.5$
3. $\frac{\partial h_1}{\partial z_1} = h_1(1 - h_1) = 0.5498 \cdot (1 - 0.5498)$
4. $\frac{\partial z_1}{\partial w_{11}} = x_1 = 1$

Step 3 — Multiply everything

$$\begin{aligned}\frac{\partial L}{\partial w_{11}} &= \delta_3 \cdot v_1 \cdot h_1(1 - h_1) \cdot x_1 \\ &= -0.064 \cdot 0.5 \cdot 0.5498 \cdot 0.4502 \cdot 1\end{aligned}$$

step by step.

We have:

$$\frac{\partial L}{\partial w_{11}} = \delta_3 \cdot v_1 \cdot h_1(1 - h_1) \cdot x_1$$

with values:

$$\delta_3 = -0.064, v_1 = 0.5, h_1 = 0.5498, x_1 = 1$$

Step 1 — Compute $h_1(1 - h_1)$ sigmoid derivative

$$\begin{aligned}h_1(1 - h_1) &= 0.5498 \cdot (1 - 0.5498) = 0.5498 \cdot 0.4502 \\ 0.5498 \cdot 0.4502 &\approx 0.2471\end{aligned}$$

✅ Step 1 done.

Step 2 — Multiply by v_1

$$v_1 \cdot h_1(1 - h_1) = 0.5 \cdot 0.2471 \approx 0.12355$$

Step 3 — Multiply by δ_3

$$\delta_3 \cdot (v_1 \cdot h_1(1 - h_1)) = -0.064 \cdot 0.12355 \approx -0.00791$$

Step 4 — Multiply by x_1

$$-0.00791 \cdot 1 = -0.00791$$

✅ Done. That is:

$$\frac{\partial L}{\partial w_{11}} \approx -0.00791$$

For w_{12} (weight from $x_2 \rightarrow h_1$)

Formula:

$$\frac{\partial L}{\partial w_{12}} = \delta_3 \cdot v_1 \cdot h_1(1 - h_1) \cdot x_2$$

From forward pass:

$$x_2 = 0$$

Step 1 — Plug numbers

$$\delta_3 \cdot v_1 \cdot h_1(1 - h_1) \cdot x_2 = -0.064 \cdot 0.5 \cdot 0.2471 \cdot 0 = 0$$

✓ So:

$$\frac{\partial L}{\partial w_{12}} = 0$$

2 For b_1 (bias of hidden neuron H1)

Formula:

$$\frac{\partial L}{\partial b_1} = \delta_3 \cdot v_1 \cdot h_1(1 - h_1) \cdot \frac{\partial z_1}{\partial b_1}$$

- $z_1 = w_{11}x_1 + w_{12}x_2 + b_1$
- $\frac{\partial z_1}{\partial b_1} = 1$

Step 1 — Plug numbers

$$\begin{aligned} \frac{\partial L}{\partial b_1} &= -0.064 \cdot 0.5 \cdot 0.2471 \cdot 1 \\ &= -0.00791 \end{aligned}$$

✓ Same as w_{11} except multiplied by $x_1 = 1$.

✓ Summary Hidden Gradients (H1)

Parameter Gradient

$$w_{11} \quad -0.00791$$

Parameter Gradient

$$w_{12} = 0$$

$$b_1 = -0.00791$$

- These gradients:

$$w_{11} = -0.00791, w_{12} = 0, b_1 = -0.00791$$

- Are all related to **H1**, which receives inputs from **x₁** and **x₂**.

let's move to **H2** step by step.

◆ Step 1: Compute hidden error term for H2

For hidden neuron h_2 , the error term is:

$$\delta_2 = \delta_3 \cdot v_2 \cdot h_2(1 - h_2)$$

We already have:

- $\delta_3 = -0.064$
- $v_2 = 0.6$
- $h_2 = 0.5987$

◆ Step 2: Compute sigmoid derivative

$$\begin{aligned} h_2(1 - h_2) &= 0.5987(1 - 0.5987) \\ &= 0.5987 \times 0.4013 \\ &\approx 0.2405 \end{aligned}$$

◆ Step 3: Multiply everything

$$\delta_2 = -0.064 \cdot 0.6 \cdot 0.2405$$

First:

$$-0.064 \cdot 0.6 = -0.0384$$

Then:

$$-0.0384 \cdot 0.2405 \approx -0.00923$$

✅ Final hidden error for H2

$$\delta_2 \approx -0.00923$$

◆ Step 4: Gradients for H2 weights

Now use:

$$\begin{aligned}\frac{\partial L}{\partial w_{21}} &= \delta_2 \cdot x_1 \\ \frac{\partial L}{\partial w_{22}} &= \delta_2 \cdot x_2 \\ \frac{\partial L}{\partial b_2} &= \delta_2\end{aligned}$$

We know:

- $x_1 = 1$
- $x_2 = 0$

◆ Compute them

$$\begin{aligned}\frac{\partial L}{\partial w_{21}} &= -0.00923 \cdot 1 = -0.00923 \\ \frac{\partial L}{\partial w_{22}} &= -0.00923 \cdot 0 = 0 \\ \frac{\partial L}{\partial b_2} &= -0.00923\end{aligned}$$

✅ Final H2 Gradients

Parameter Gradient

$$w_{21} \quad -0.00923$$

$$w_{22} \quad 0$$

$$b_2 \quad -0.00923$$

Now your **entire hidden layer backprop is complete**

◆ Next Step = Weight Update (Gradient Descent)

Now we actually **update all parameters** using:

$$\text{new weight} = \text{old weight} - \eta \cdot \text{gradient}$$

Where:

- η = learning rate
- gradient = what we computed

◆ What needs to be updated?

1 Hidden → Output

- v_1
- v_2
- b_3

2 Input → Hidden (H1)

- w_{11}, w_{12}, b_1

3 Input → Hidden (H2)

- w_{21}, w_{22}, b_2

So now we update **all 9 parameters**.

◆ After Updating

Once weights are updated:

1. Do another **forward pass**
2. Compute new prediction
3. Compute new loss
4. Repeat (this is one training iteration / one step of gradient descent)

🔥 Big Picture

Forward pass → Compute loss → Backprop → Update weights

That entire sequence = **one training step**

Perfect 👍 let's update **everything step by step** using

$$\eta = 0.1$$

Update rule:

$$\text{new} = \text{old} - 0.1 \cdot \text{gradient}$$

◆ 1 Update Hidden → Output Weights

We previously had:

- $\frac{\partial L}{\partial v_1} = -0.0352$
- $\frac{\partial L}{\partial v_2} = -0.0383$
- $\frac{\partial L}{\partial b_3} = -0.064$

Old values:

- $v_1 = 0.5$
- $v_2 = 0.6$
- $b_3 = 0.2$

Update v_1

$$\begin{aligned} v_1^{new} &= 0.5 - 0.1(-0.0352) \\ &= 0.5 + 0.00352 \\ &= 0.50352 \end{aligned}$$

Update v_2

$$\begin{aligned} v_2^{new} &= 0.6 - 0.1(-0.0383) \\ &= 0.6 + 0.00383 \\ &= 0.60383 \end{aligned}$$

Update b_3

$$\begin{aligned} b_3^{new} &= 0.2 - 0.1(-0.064) \\ &= 0.2 + 0.0064 \\ &= 0.2064 \end{aligned}$$

◆ 2 Update Input → Hidden (H1)

Gradients:

- $w_{11} = -0.00791$
- $w_{12} = 0$
- $b_1 = -0.00791$

Old:

- $w_{11} = 0.1$
- $w_{12} = 0.2$
- $b_1 = 0.1$

Update w_{11}

$$0.1 - 0.1(-0.00791) = 0.100791$$

Update w_{12}

$$0.2 - 0.1(0) = 0.2$$

Update b_1

$$0.1 - 0.1(-0.00791) = 0.100791$$

◆ **3 Update Input → Hidden (H2)**

Gradients:

- $w_{21} = -0.00923$
- $w_{22} = 0$
- $b_2 = -0.00923$

Old:

- $w_{21} = 0.3$
- $w_{22} = 0.4$
- $b_2 = 0.1$

Update w_{21}

$$0.3 - 0.1(-0.00923) = 0.300923$$

Update w_{22}

$$0.4 - 0.1(0) = 0.4$$

Update b_2

$$0.1 - 0.1(-0.00923) = 0.100923$$

✅ **Final Updated Parameters**

Hidden → Output

- $v_1 = 0.50352$
- $v_2 = 0.60383$
- $b_3 = 0.2064$

Input → Hidden

- $w_{11} = 0.100791$
- $w_{12} = 0.2$
- $w_{21} = 0.300923$
- $w_{22} = 0.4$
- $b_1 = 0.100791$
- $b_2 = 0.100923$

 You just completed ONE full training step.